Dokumentacja Optymalizacji Modelu CNN dla Zbioru Danych MNIST

Dokumentacja Techniczna PyTorch

27 kwietnia 2026

Spis treści

1 Wstęp **2**

2 Zadanie 1: Regularyzacja L2 (Weight Decay) **2**

3 Zadanie 2: Różnica między Validation Loss a Train Loss **2**

4 Zadanie 3: Optymalizacja modelu CNN ($> 99\%$ Accuracy) **2**

5 Zadanie 4: Macierz Błędów (Confusion Matrix) **3**

1 Wstęp

Niniejsza dokumentacja techniczna opisuje proces modyfikacji i strojenia (ang. *tuning*) Konwolucyjnej Sieci Neuronowej (CNN) służącej do klasyfikacji obrazów ze zbioru MNIST. Celem podjętych działań było zastosowanie technik zapobiegających przeuczeniu, wyjaśnienie anomalii funkcji straty, osiągnięcie skuteczności (ang. *accuracy*) na poziomie powyżej 99% na zbiorze testowym oraz wizualizacja pomyłek modelu za pomocą macierzy błędów.

2 Zadanie 1: Regularyzacja L2 (Weight Decay)

Aby zapobiec zjawisku przeuczenia w modelach głębokich (np. klasie `Deep`), wprowadzono regularyzację L2. W bibliotece PyTorch realizuje się to poprzez dodanie parametru `weight_decay` do optymalizatora.

Regularyzacja L2 dodaje do standardowej funkcji straty $\mathcal{L}_0$ człon karzący za zbyt duże wagi, co można zapisać wzorem:

$$\mathcal{L} = \mathcal{L}_0 + \lambda \sum_i w_i^2$$

gdzie λ to współczynnik siły regularyzacji (w naszym kodzie ustalony na np. 10^{-4}), a w_i to poszczególne wagi modelu. Ograniczenie wielkości wag zapobiega nadmiernemu dopasowywaniu się sieci do szumu obecnego w danych treningowych.

3 Zadanie 2: Różnica między Validation Loss a Train Loss

W początkowej fazie trenowania zauważono anomalię: strata na zbiorze walidacyjnym była niższa niż strata na zbiorze treningowym. Jest to bezpośredni wynik zastosowania warstwy **Dropout**.

Warstwa Dropout podczas treningu wyzerowuje losowo wybraną frakcję neuronów z prawdopodobieństwem p. Operacja ta sztucznie "upośledza"sieć na etapie uczenia, co podnosi wartość *Train Loss*. Podczas ewaluacji (wywołanie `model.eval()`) na zbiorze walidacyjnym i testowym, mechanizm Dropout jest wyłączany. Model korzysta ze wszystkich wyuczonych wag jednocześnie, co skutkuje znacznie wyższą skutecznością i niższym błędem (*Validation Loss*).

4 Zadanie 3: Optymalizacja modelu CNN ($>$ 99% Accuracy)

Aby przekroczyć próg 99% poprawnych klasyfikacji na zbiorze testowym, wprowadzono następujące zmiany w architekturze `TunedCNN`:

- **Zwiększenie liczby filtrów konwolucyjnych:** Pierwsza warstwa wyodrębnia 16 map cech (zamiast 2), a druga 32 (zamiast 4). Zwiększa to pojemność informacyjną modelu.

- **Zmiana klasyfikatora liniowego:** Warstwy w pełni połączone (Dense) zostały dostosowane do rozmiaru tensora po operacjach `MaxPool2d`. Liczba neuronów w warstwie ukrytej wynosi 128.
- **Zmiana optymalizatora:** Przejście ze stochastycznego spadku gradientu (SGD) na **Adam** (Adaptive Moment Estimation) ze współczynnikiem uczenia $\alpha = 0.001$, co znacząco przyspieszyło zbieżność.

5 Zadanie 4: Macierz Błędów (Confusion Matrix)

Do szczegółowej oceny jakości klasyfikacji poszczególnych cyfr wygenerowano macierz błędów C, w której każdy element $C_{i,j}$ jest zdefiniowany jako liczba obserwacji z prawdziwej klasy i, które zostały sklasyfikowane przez model do klasy j.

Dla problemu 10-klasowego (cyfry 0-9), diagonalna $C_{i,i}$ reprezentuje poprawnie rozpoznane próbki (*True Positives*). Pozostałe elementy reprezentują pomyłki modelu. Przykładowo, element $C_{3,5}$ przechowuje informację, ile razy sieć zobaczyła na obrazku cyfrę "3", ale błędnie uznała ją za "5". Wizualizacja macierzy za pomocą biblioteki `scikit-learn` pozwala szybko zidentyfikować cyfry najczęściej mylone ze sobą, co zwykle dotyczy par $(4, 9)$ oraz $(7, 1)$ z powodu podobieństwa ich odręcznych kształtów.